TDVI: Inteligencia Artificial

Transfer learning & Encoders/Decoders

Licenciatura en Tecnología Digital

Agenda

1. **Motivación**
2. Upsampling
   a. Interpolación
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. SAM
   e. FAN
4. Bibliografía

Problemas de imagen a imagen

- Hasta ahora vimos problemas que mapeaban una imagen a un número real o etiqueta
- En computación gráfica las tareas pueden requerir outputs más complejos:
  - imágenes
  - volúmenes
  - mallas 3D
  - etc

Problemas de imagen a imagen

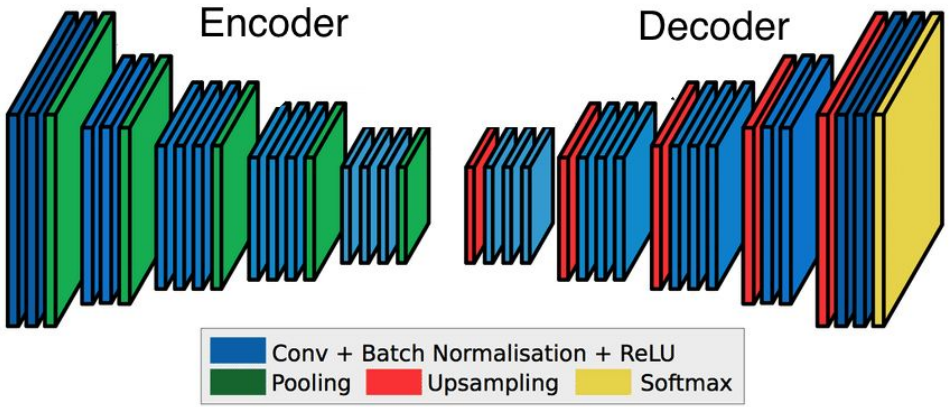
Problemas de imagen a imagen

winter Yosemite → summer Yosemite

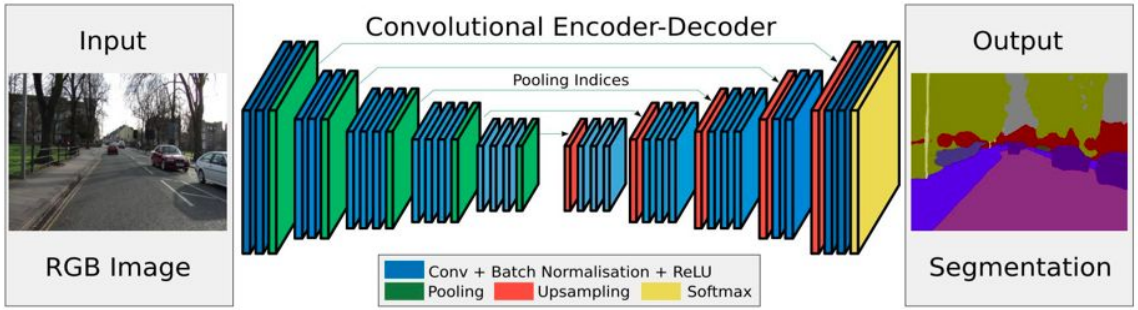
summer Yosemite → winter Yosemite

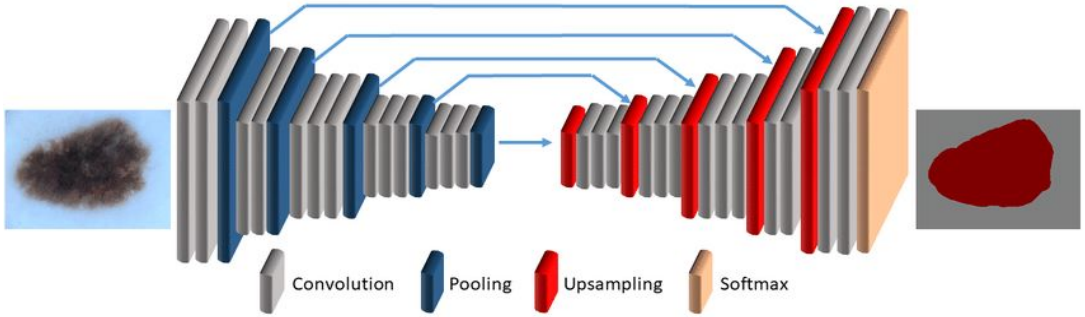
Encoder–Decoder

- **Encoder:** Convierte los datos de entrada en algo compacto y abstracto.
  - Una representación vectorial que “vive” en el espacio latente de la red neuronal.
- **Decoder:** Convierte el vector nuevamente al espacio original.

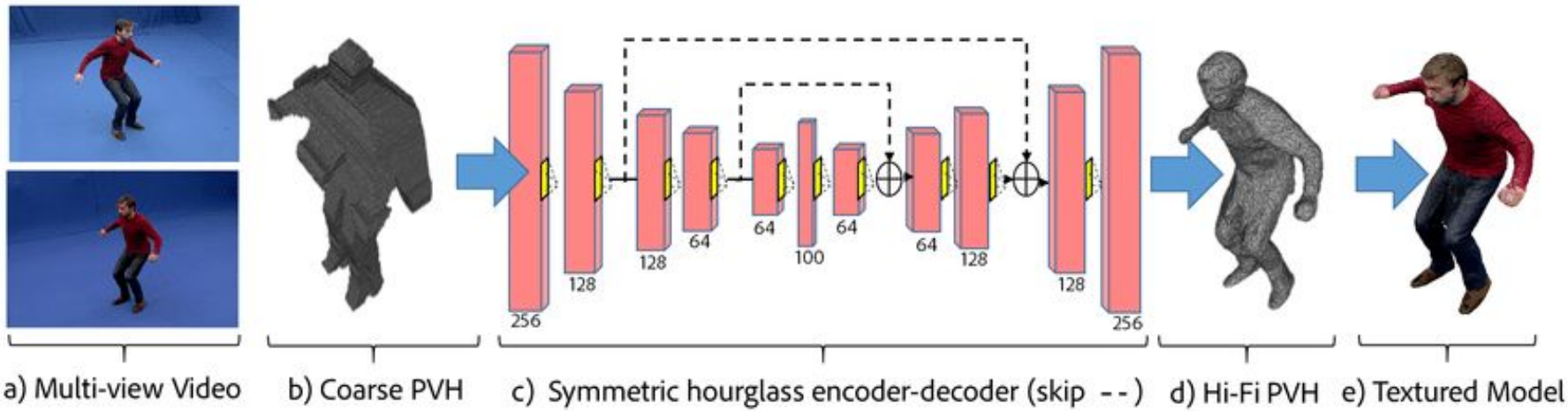


Segmentación

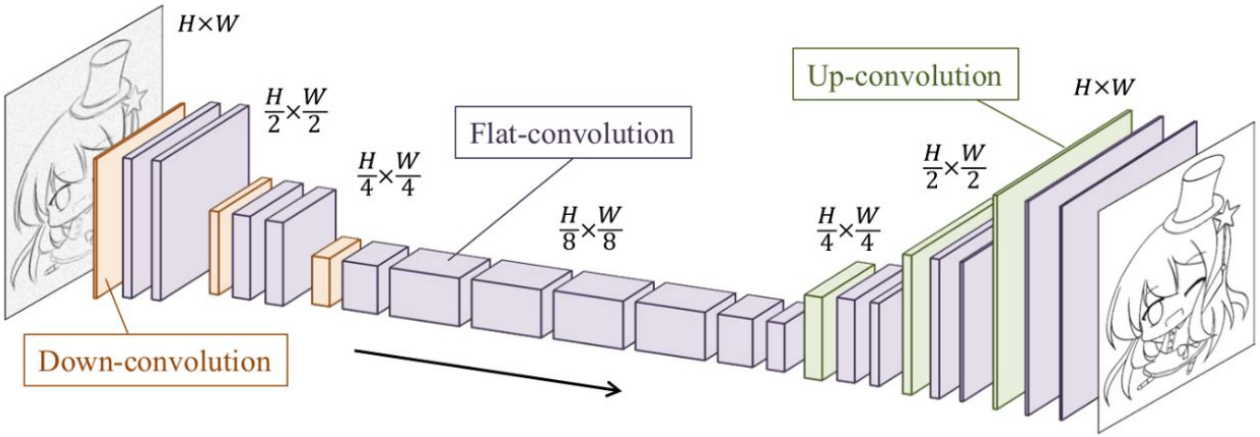




Video a malla 3D



Operadores



Down-convolutions

Flat-convolutions

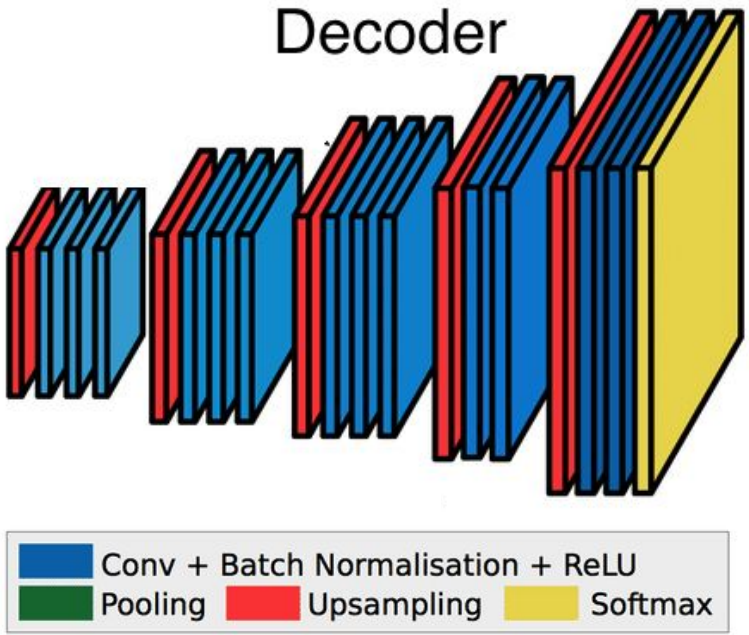
Up-convolutions
(transposed convolutions)

Agenda

1. Motivación
2. **Upsampling**
    a. Interpolación
    b. Convoluciones transpuestas
3. Segmentación de imágenes
    a. U-Net
    b. Yolo
    c. R-CNN
    d. SAM
    e. FAN
4. Bibliografía

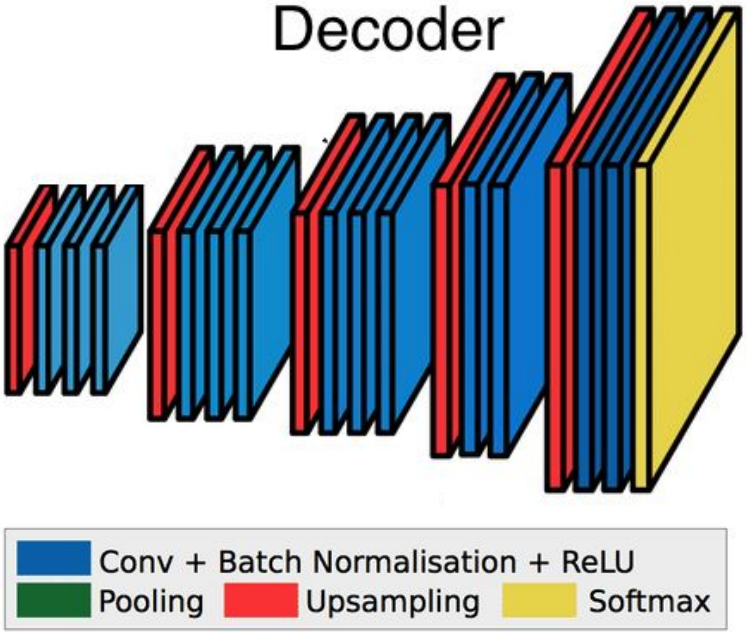
¿Up-Sampling?

- Hasta ahora sabemos:
  - Mantener la resolución:
    - Padding, y stride=1
  - Disminuir la resolución:
    - Pooling



¿Up-Sampling?

- Hasta ahora sabemos:
  - Mantener la resolución:
    - Padding, y stride=1
  - Disminuir la resolución:
    - Pooling
- ¿Cómo incrementamos?
  - Interpolación
  - Convoluciones transpuestas



Agenda

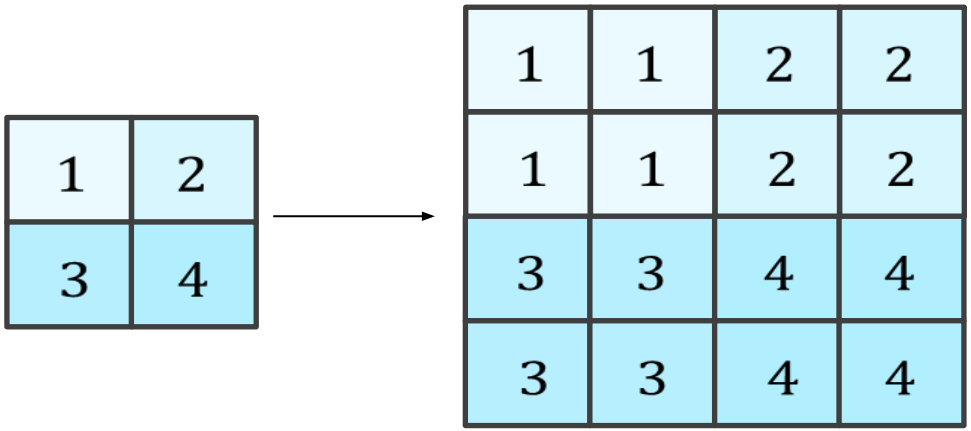
1. Motivación
2. Upsampling
   a. **Interpolación**
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. SAM
   e. FAN
4. Bibliografía

Interpolación

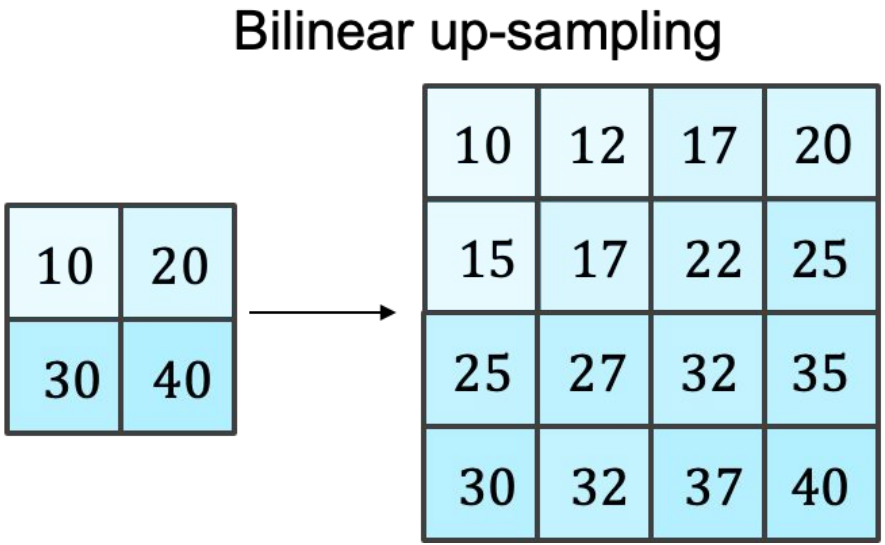
- En éste método se suele utilizar interpolación de tipo:
  - nearest (vecinos más cercanos)
  - bilineal
- Se computan nuevos valores a partir de una fórmula

Nearest

Nearest neighbor up-sampling



Bilinear

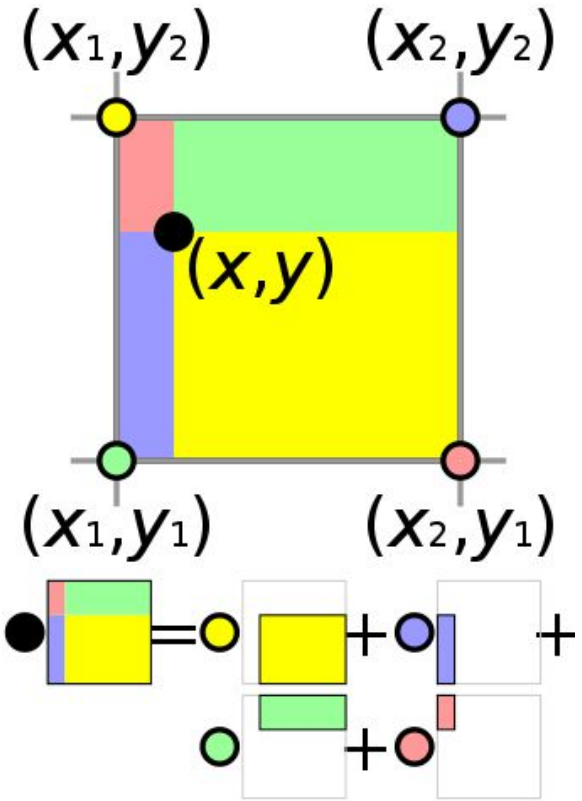

Bilinear up-sampling
10 20
30 40
10 12 17 20
15 17 22 25
25 27 32 35
30 32 37 40

Bilinear

$Q_{11} = (x_1, y_1)$, $Q_{12} = (x_1, y_2)$, $Q_{21} = (x_2, y_1)$, and $Q_{22} = (x_2, y_2)$.

$$f(x, y_1) = \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}),$$
$$f(x, y_2) = \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}).$$

$$f(x, y) = \frac{y_2 - y}{y_2 - y_1} f(x, y_1) + \frac{y - y_1}{y_2 - y_1} f(x, y_2)$$

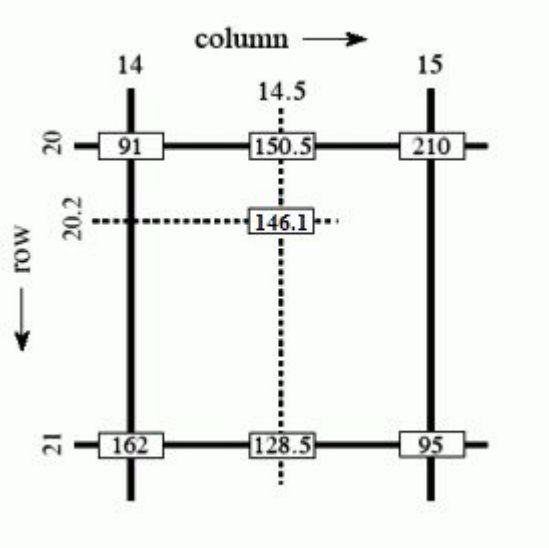


Bilinear

$$I_{20,14.5} = \frac{15 - 14.5}{15 - 14} \cdot 91 + \frac{14.5 - 14}{15 - 14} \cdot 210 = 150.5,$$

$$I_{21,14.5} = \frac{15 - 14.5}{15 - 14} \cdot 162 + \frac{14.5 - 14}{15 - 14} \cdot 95 = 128.5,$$

$$I_{20.2,14.5} = \frac{21 - 20.2}{21 - 20} \cdot 150.5 + \frac{20.2 - 20}{21 - 20} \cdot 128.5 = 146.1.$$



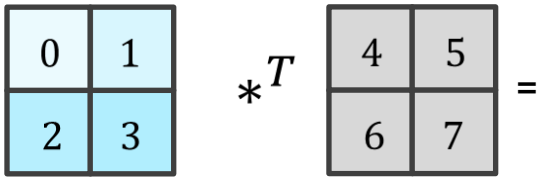
Agenda

1. Motivación
2. Upsampling
   a. Interpolación
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. SAM
   e. FAN
4. Bibliografía

Convolución transpuesta

- Convoluciones en sentido opuesto: deconvolución o convolución transpuesta.
- Permite a la red aprender un proceso de up-sampleo en vez de basarse en una ecuación matematica fija. It allows the network to **learn the up-sampling process** instead of relying on a fixed mathematical formula.

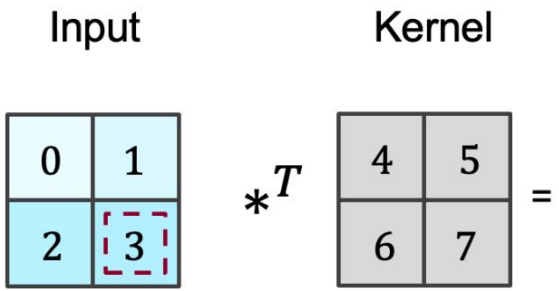
Input Kernel



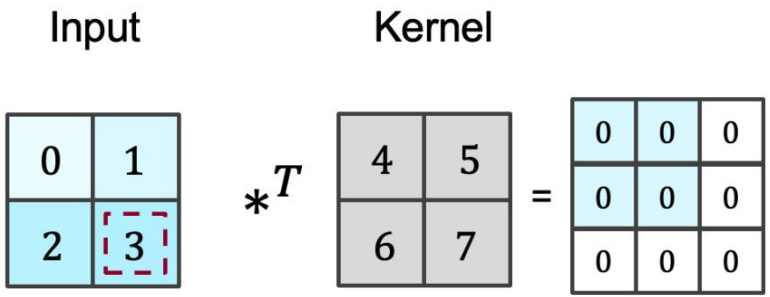
Convolución transpuesta

- Convoluciones en sentido opuesto: deconvolución o convolución transpuesta.
- Permite a la red aprender un proceso de up-sampleo en vez de basarse en una ecuación matemática fija.

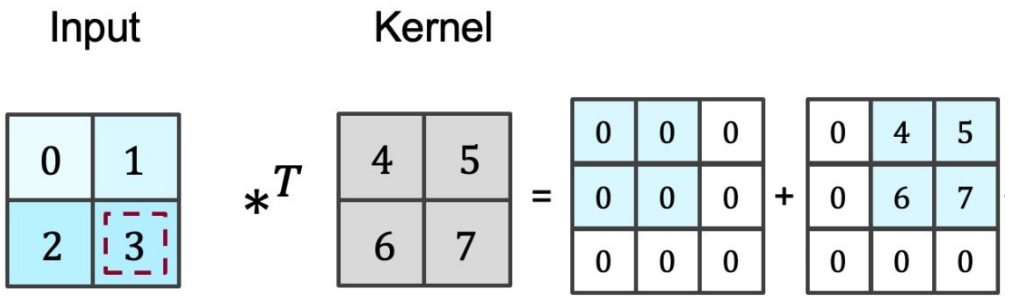
Ejemplo de convolución transpuesta



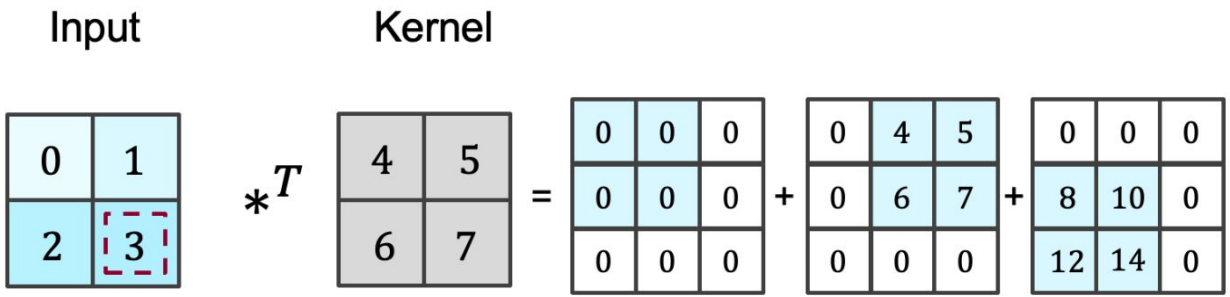
Ejemplo de convolución transpuesta



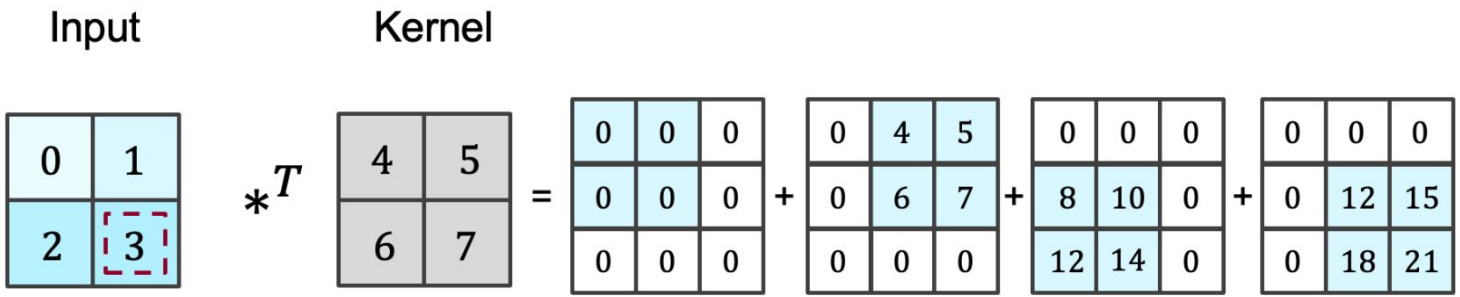
Ejemplo de convolución transpuesta



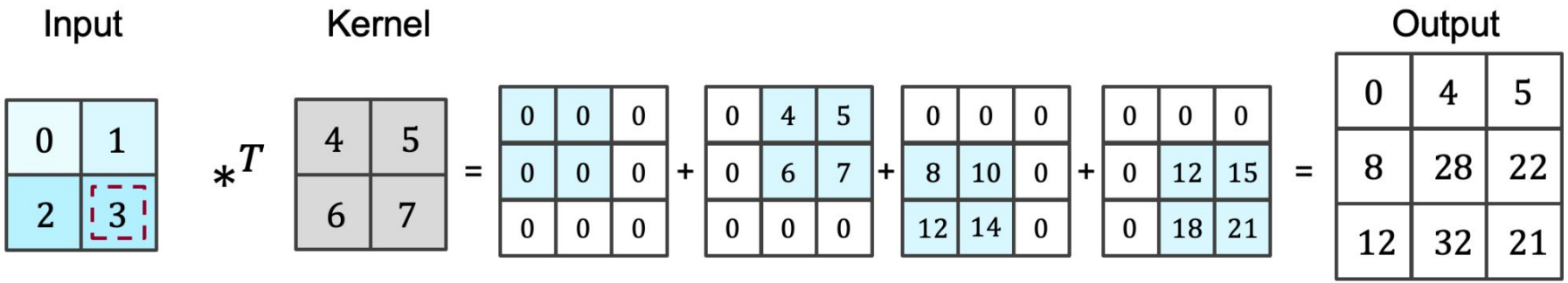
Ejemplo de convolución transpuesta



Ejemplo de convolución transpuesta



Ejemplo de convolución transpuesta



Convolución transpuesta

- Podemos parametrizar las convoluciones transpuestas con **stride** y **padding**.
- Padding para las convoluciones transpuestas significa que las filas y columnas se **eliminan**.

Ejemplo de código

```python
import torch
import torch.nn as nn

#Ejemplo 1
# With square kernels and equal stride
m = nn.ConvTranspose2d(16, 33, 3, stride=2)
input = torch.randn(20, 16, 50, 100)

output = m(input)
print(output.size())

#Ejemplo 2
# exact output size can be also specified as an argument
input = torch.randn(1, 16, 12, 12)
downsample = nn.Conv2d(16, 16, 4, stride=2, padding=1)
upsample = nn.ConvTranspose2d(16, 16, 4, stride=2, padding=1)
h = downsample(input)

print(h.size())
output = upsample(h)

print(output.size())
```

Agenda

1. Motivación
2. Upsampling
   a. Interpolación
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. SAM
   e. FAN
4. Bibliografía

U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

Computer Science Department and BIOSS Centre for Biological Signalling Studies,
University of Freiburg, Germany
ronneber@informatik.uni-freiburg.de,
WWW home page: http://lmb.informatik.uni-freiburg.de/

Abstract. There is large consent that successful training of deep networks requires many thousand annotated training samples. In this paper, we present a network and training strategy that relies on the strong use of data augmentation to use the available annotated samples more efficiently. The architecture consists of a contracting path to capture context and a symmetric expanding path that enables precise localization. We show that such a network can be trained end-to-end from very few images and outperforms the prior best method (a sliding-window convolutional network) on the ISBI challenge for segmentation of neuronal structures in electron microscopic stacks. Using the same network trained on transmitted light microscopy images (phase contrast and DIC) we won the ISBI cell tracking challenge 2015 in these categories by a large margin. Moreover, the network is fast. Segmentation of a 512x512 image takes less than a second on a recent GPU. The full implementation (based on Caffe) and the trained networks are available at http://lmb.informatik.uni-freiburg.de/people/ronneber/u-net.

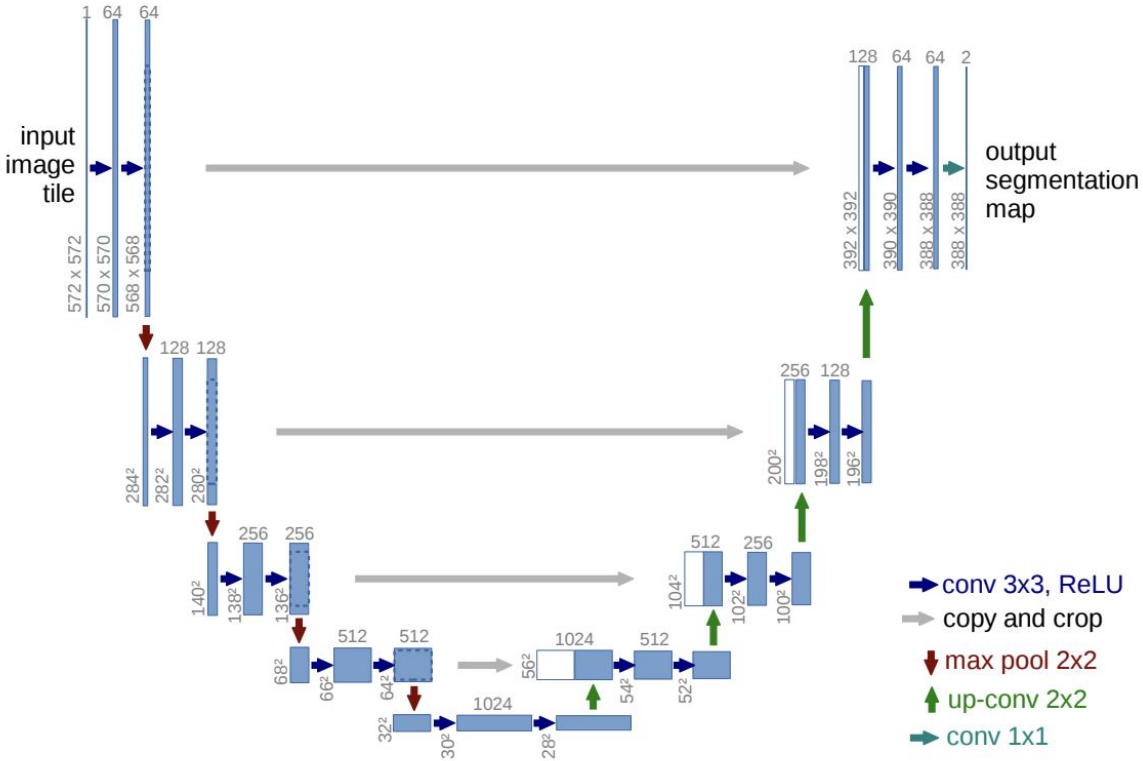


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

U-Net

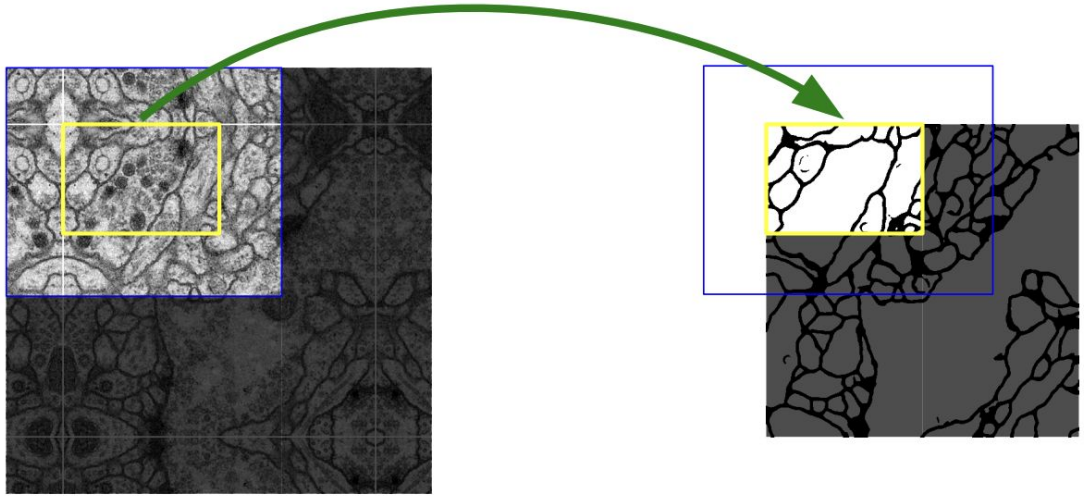


Fig. 2. Overlap-tile strategy for seamless segmentation of arbitrary large images (here segmentation of neuronal structures in EM stacks). Prediction of the segmentation in the yellow area, requires image data within the blue area as input. Missing input data is extrapolated by mirroring

U-Net

- La última capa es una convolución 1x1 de 64 feature maps, a la cantidad de clases
- Soft-max en la capa final
- Cross entropy loss

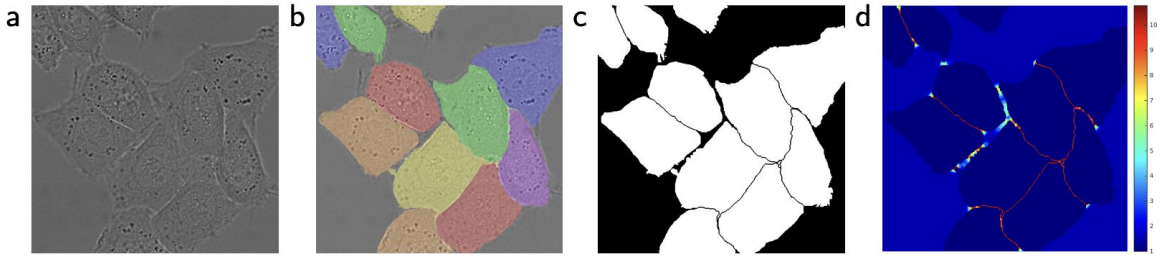


Fig. 3. HeLa cells on glass recorded with DIC (differential interference contrast) microscopy. (**a**) raw image. (**b**) overlay with ground truth segmentation. Different colors indicate different instances of the HeLa cells. (**c**) generated segmentation mask (white: foreground, black: background). (**d**) map with a pixel-wise loss weight to force the network to learn the border pixels.

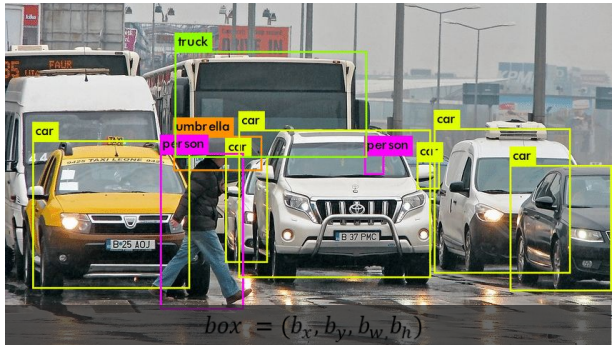
Agenda

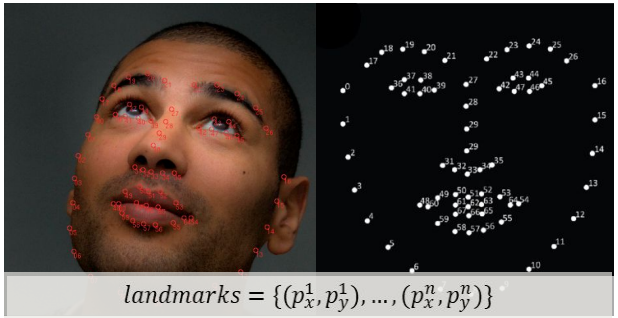
1. Motivación
2. Upsampling
   a. Interpolación
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. FAN
4. Bibliografía

Localización y detección

Existen distintos métodos

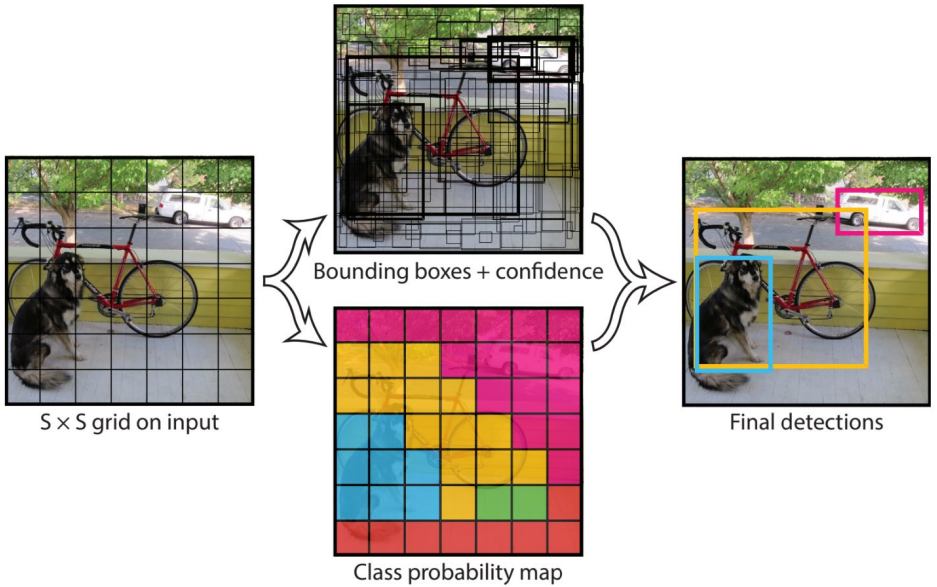
- **Detección de bounding box:**
  - Detecta la sección de la imagen donde se encuentra el objeto
- **Detección de landmarks:**
  - Detecta puntos correspondientes a una forma o características particulares de un objeto.





Detección de bounding box con Yolo

- Divide la imagen de entrada en un grilla (píxeles más grandes)
- Predice objetos y atributos para cada celda
- Utiliza supresión de no-máximos para eliminar predicciones sobrepuestas **previo a la predicción final**



You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*†, Ross Girshick¶, Ali Farhadi*†
University of Washington*, Allen Institute for AI†, Facebook AI Research¶
http://pjreddie.com/yolo/

Abstract

We present YOLO, a new approach to object detection. Prior work on object detection repurposes classifiers to perform detection. Instead, we frame object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. Since the whole detection pipeline is a single network, it can be optimized end-to-end directly on detection performance.

Our unified architecture is extremely fast. Our base YOLO model processes images in real-time at 45 frames per second. A smaller version of the network, Fast YOLO, processes an astounding 155 frames per second while still achieving double the mAP of other real-time detectors. Compared to state-of-the-art detection systems, YOLO makes more localization errors but is less likely to predict false positives on background. Finally, YOLO learns very general representations of objects. It outperforms other detection methods, including DPM and R-CNN, when generalizing from natural images to other domains like artwork.

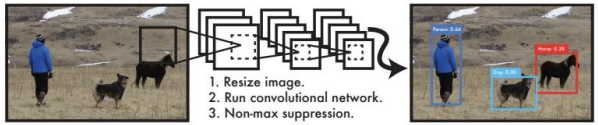


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448 × 448, (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

methods to first generate potential bounding boxes in an image and then run a classifier on these proposed boxes. After classification, post-processing is used to refine the bounding boxes, eliminate duplicate detections, and rescore the boxes based on other objects in the scene [13]. These complex pipelines are slow and hard to optimize because each individual component must be trained separately.

We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. Using our system, you only look once (YOLO) at an image to predict what objects are

Arquitectura de YOLO

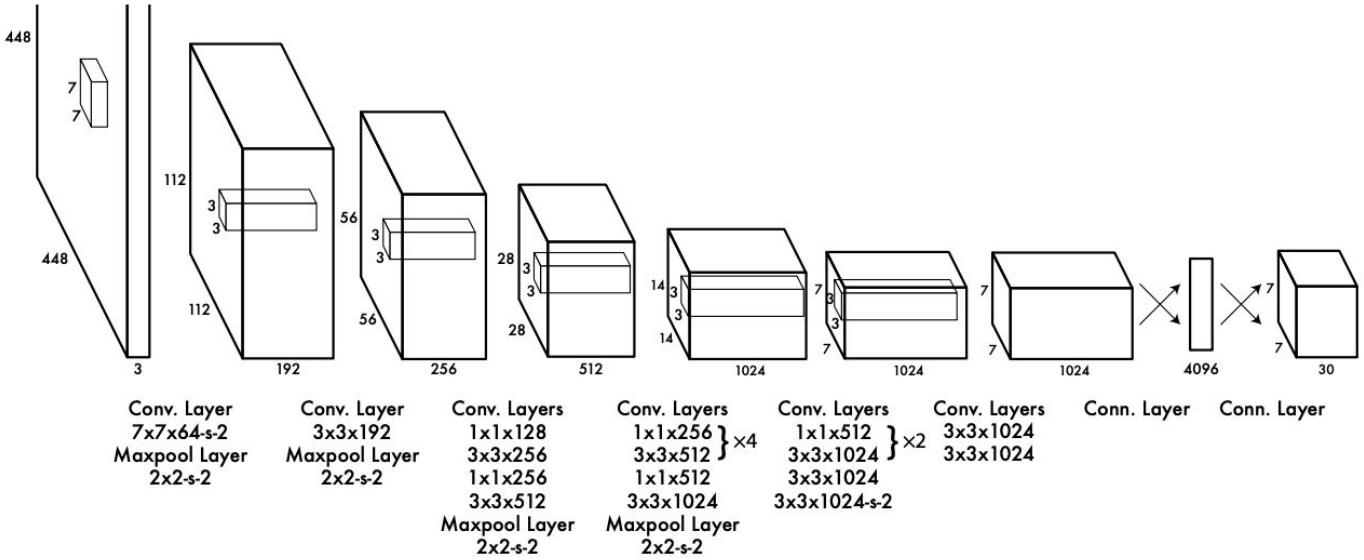


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

Yolo Loss

loss function:

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

Tamaño del bounding box

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

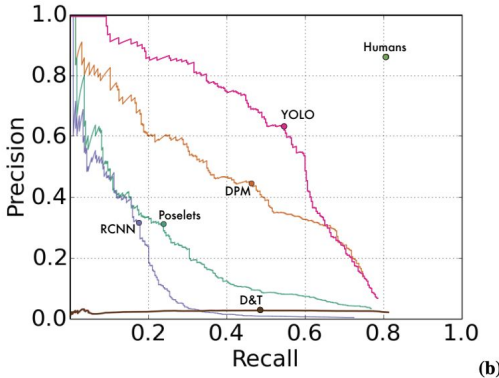
$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2$$

Confianza del bounding box

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2$$

$$+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

Probabilidad de la clase



(a) Picasso Dataset precision-recall curves.

	VOC 2007	Picasso		People-Art
	AP	AP	Best F_1	AP
YOLO	**59.2**	**53.3**	**0.590**	**45**
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 5: Generalization results on Picasso and People-Art datasets.

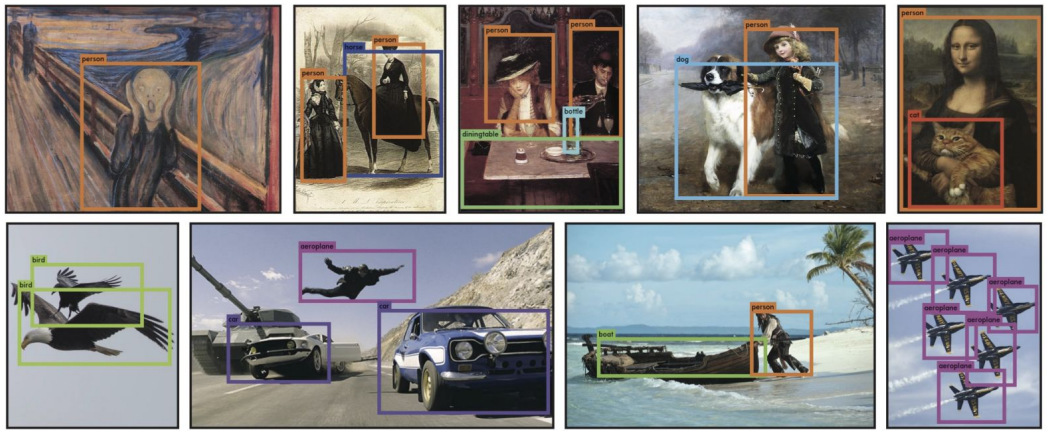


Figure 6: Qualitative Results. YOLO running on sample artwork and natural images from the internet. It is mostly accurate although it does think one person is an airplane.

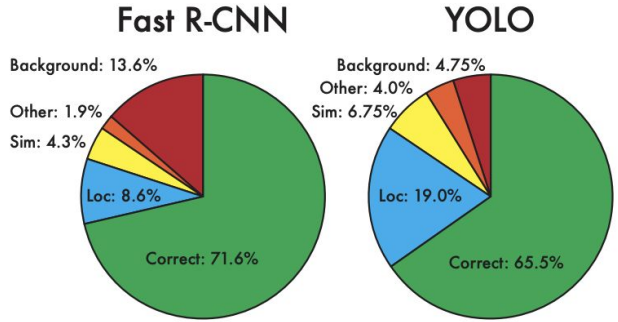


Figure 4: Error Analysis: Fast R-CNN vs. YOLO These charts show the percentage of localization and background errors in the top N detections for various categories (N = # objects in that category).

Agenda

1. Motivación
2. Upsampling
   a. Interpolación
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. FAN
4. Bibliografía

Rich feature hierarchies for accurate object detection and semantic segmentation

Tech report (v5)

Ross Girshick Jeff Donahue Trevor Darrell Jitendra Malik
UC Berkeley
{rbg,jdonahue,trevor,malik}@eecs.berkeley.edu

Abstract

Object detection performance, as measured on the canonical PASCAL VOC dataset, has plateaued in the last few years. The best-performing methods are complex ensemble systems that typically combine multiple low-level image features with high-level context. In this paper, we propose a simple and scalable detection algorithm that improves mean average precision (mAP) by more than 30% relative to the previous best result on VOC 2012—achieving a mAP of 53.3%. Our approach combines two key insights: (1) one can apply high-capacity convolutional neural networks (CNNs) to bottom-up region proposals in order to localize and segment objects and (2) when labeled training data is scarce, supervised pre-training for an auxiliary task, followed by domain-specific fine-tuning, yields a significant performance boost. Since we combine region proposals with CNNs, we call our method R-CNN: *Regions with CNN features. We also compare R-CNN to OverFeat, a recently proposed sliding-window detector based on a similar CNN architecture. We find that R-CNN outperforms OverFeat by a large margin on the 200-class ILSVRC2013 detection dataset. Source code for the complete system is available at* http://www.cs.berkeley.edu/~rbg/rcnn.

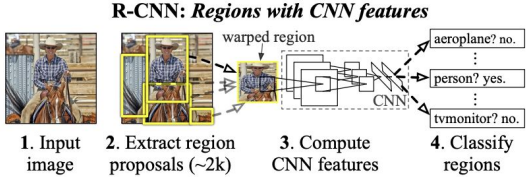


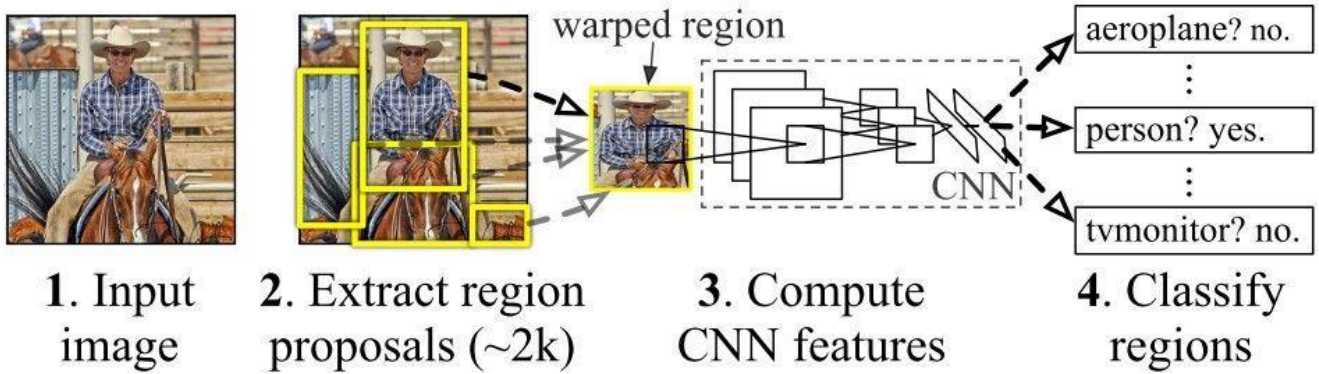
Figure 1: Object detection system overview. Our system (1) takes an input image, (2) extracts around 2000 bottom-up region proposals, (3) computes features for each proposal using a large convolutional neural network (CNN), and then (4) classifies each region using class-specific linear SVMs. R-CNN achieves a mean average precision (mAP) of **53.7% on PASCAL VOC 2010**. For comparison, [39] reports 35.1% mAP using the same region proposals, but with a spatial pyramid and bag-of-visual-words approach. The popular deformable part models perform at 33.4%. On the 200-class **ILSVRC2013 detection dataset, R-CNN's mAP is 31.4%**, a large improvement over OverFeat [34], which had the previous best result at 24.3%.

archical, multi-stage processes for computing features that are even more informative for visual recognition.

Fukushima's "neocognitron" [19], a biologically-inspired hierarchical and shift-invariant model for pattern recognition, was an early attempt at just such a process.

Detección de bounding box con R-CNN

- Genera regiones candidatas de interés y les extrae características
- Ejecuta una red de detección para encontrar el objeto más probable en cada región

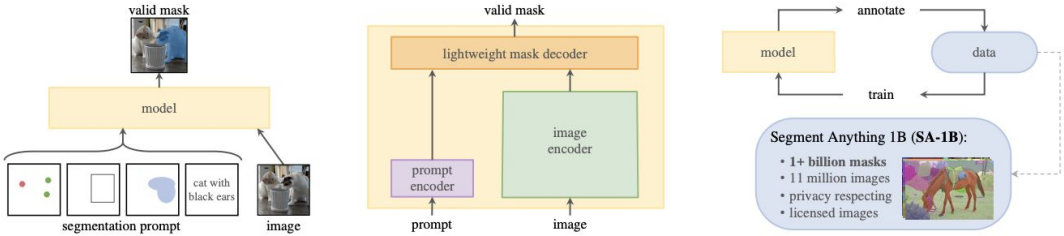


Agenda

1. Motivación
2. Upsampling
   a. Interpolación
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. SAM
   e. FAN
4. Bibliografía

Segment Anything

Alexander Kirillov[1,2,4] Eric Mintun[2] Nikhila Ravi[1,2] Hanzi Mao[2] Chloe Rolland[3] Laura Gustafson[3]
Tete Xiao[3] Spencer Whitehead Alexander C. Berg Wan-Yen Lo Piotr Dollár[4] Ross Girshick[4]
[1]project lead [2]joint first author [3]equal contribution [4]directional lead
Meta AI Research, FAIR



(a) **Task**: promptable segmentation (b) **Model**: Segment Anything Model (**SAM**) (c) **Data**: data engine (top) & dataset (bottom)

Figure 1: We aim to build a foundation model for segmentation by introducing three interconnected components: a promptable segmentation *task*, a segmentation *model* (SAM) that powers data annotation and enables zero-shot transfer to a range of tasks via prompt engineering, and a *data* engine for collecting SA-1B, our dataset of over 1 billion masks.

Abstract

We introduce the Segment Anything (SA) project: a new task, model, and dataset for image segmentation. Using our efficient model in a data collection loop, we built the largest segmentation dataset to date (by far), with over 1 ***billion*** *masks on 11M licensed and privacy respecting images. The model is designed and trained to be promptable, so it can transfer zero-shot to new image distributions and tasks. We evaluate its capabilities on numerous tasks and find that its zero-shot performance is impressive – often competitive with or even superior to prior fully supervised results. We are releasing the Segment Anything Model (SAM) and corresponding dataset (SA-1B) of 1B masks and 11M images at https://segment-anything.com to foster research into foundation models for computer vision.*

matching in some cases) fine-tuned models [10, 21]. Empirical trends show this behavior improving with model scale, dataset size, and total training compute [56, 10, 21, 51].

Foundation models have also been explored in computer vision, albeit to a lesser extent. Perhaps the most prominent illustration aligns paired text and images from the web. For example, CLIP [82] and ALIGN [55] use contrastive learning to train text and image encoders that align the two modalities. Once trained, engineered text prompts enable zero-shot generalization to novel visual concepts and data distributions. Such encoders also compose effectively with other modules to enable downstream tasks, such as image generation (*e.g.*, DALL·E [83]). While much progress has been made on vision and language encoders, computer vision includes a wide range of problems beyond this scope, and for many of these, abundant training data does not exist.

SAM (Segment Anything Model)

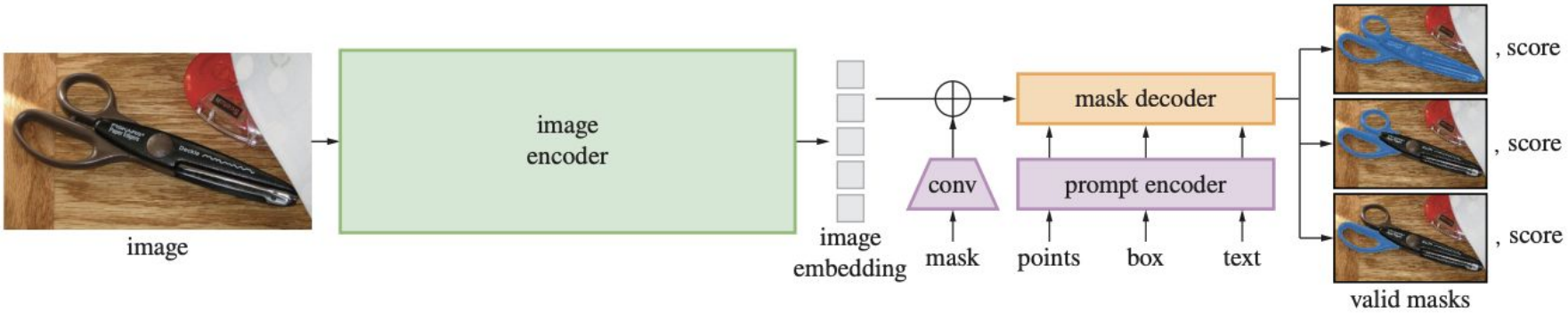


Figure 4: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

Dataset utilizado

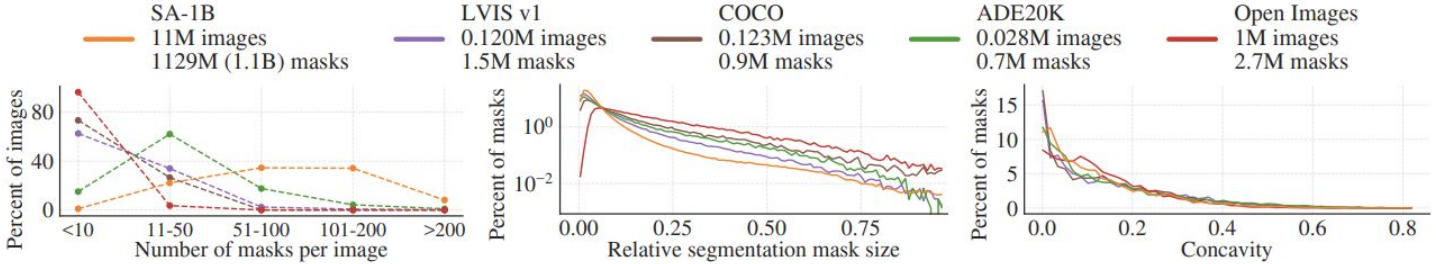


Figure 6: Dataset mask properties. The legend references the number of images and masks in each dataset. Note, that SA-1B has 11× more images and 400× more masks than the largest existing segmentation dataset Open Images [60].

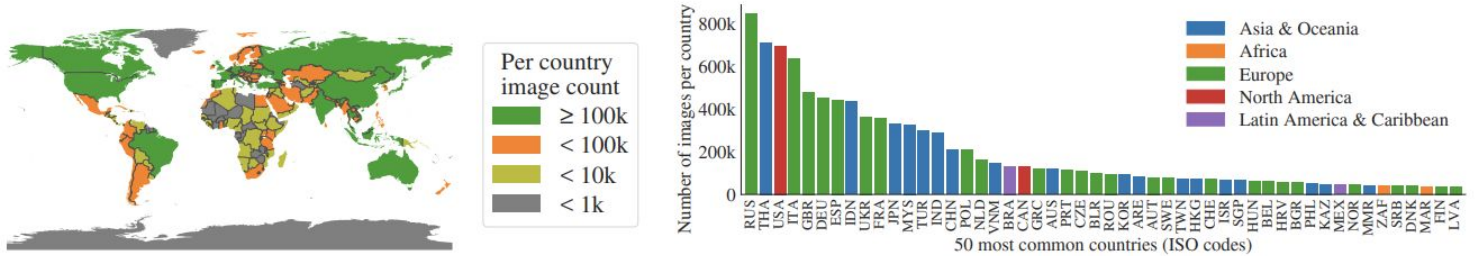


Figure 7: Estimated geographic distribution of SA-1B images. Most of the world's countries have more than 1000 images in SA-1B, and the three countries with the most images are from different parts of the world.

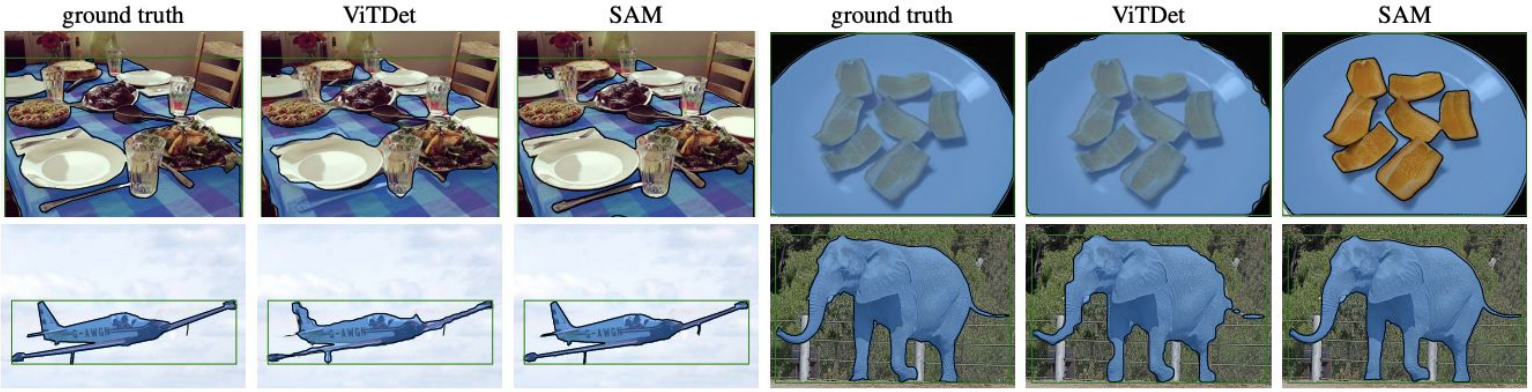


Figure 16: Zero-shot instance segmentation on LVIS v1. SAM produces higher quality masks than ViTDet. As a zero-shot model, SAM does not have the opportunity to learn specific training data biases; see top-right as an example where SAM makes a modal prediction, whereas the ground truth in LVIS is amodal given that mask annotations in LVIS have no holes.

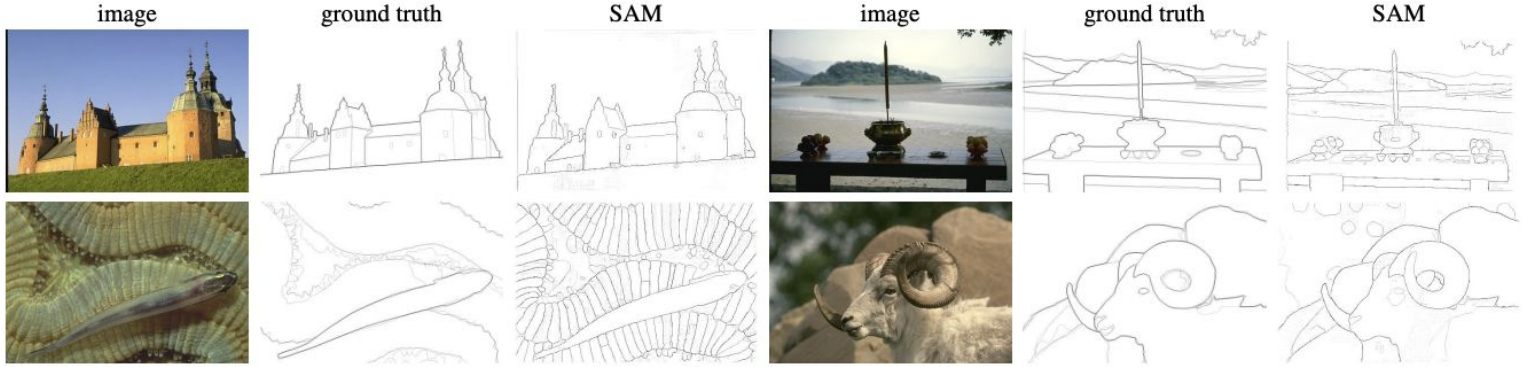


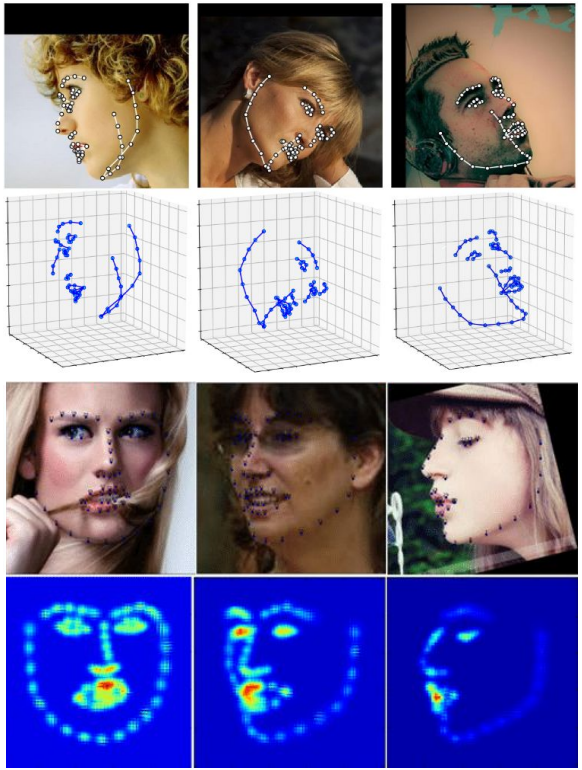
Figure 15: Additional visualizations of zero-shot edge predictions on BSDS500. Recall that SAM was not trained to predict edge maps and did not have access to BSDS images and annotations during training.

Agenda

1. Motivación
2. Upsampling
   a. Interpolación
   b. Convoluciones transpuestas
3. Segmentación de imágenes
   a. U-Net
   b. Yolo
   c. R-CNN
   d. SAM
   e. FAN
4. Bibliografía

FAN (Alineamiento de cara)

- Consiste en una serie de convoluciones y FC capas, seguido de un predictor de heatmap
- El heatmap representa la likelihood de que un landmark se encuentre en esa posición.
- La localización del landmark se encuentra tomando los valores más altos del heatmap



FAN (Alineamiento de cara)

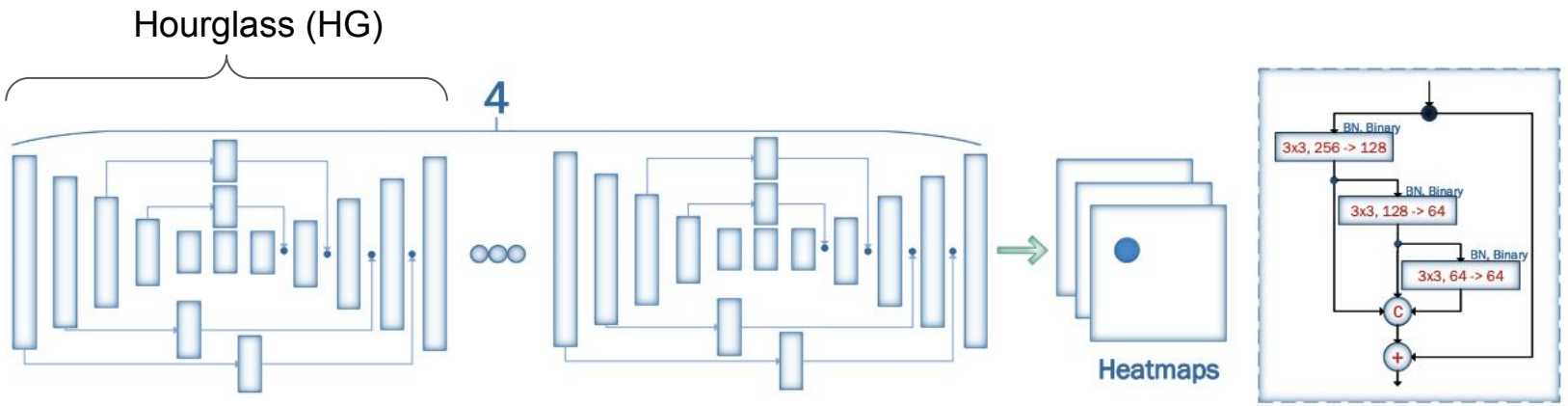
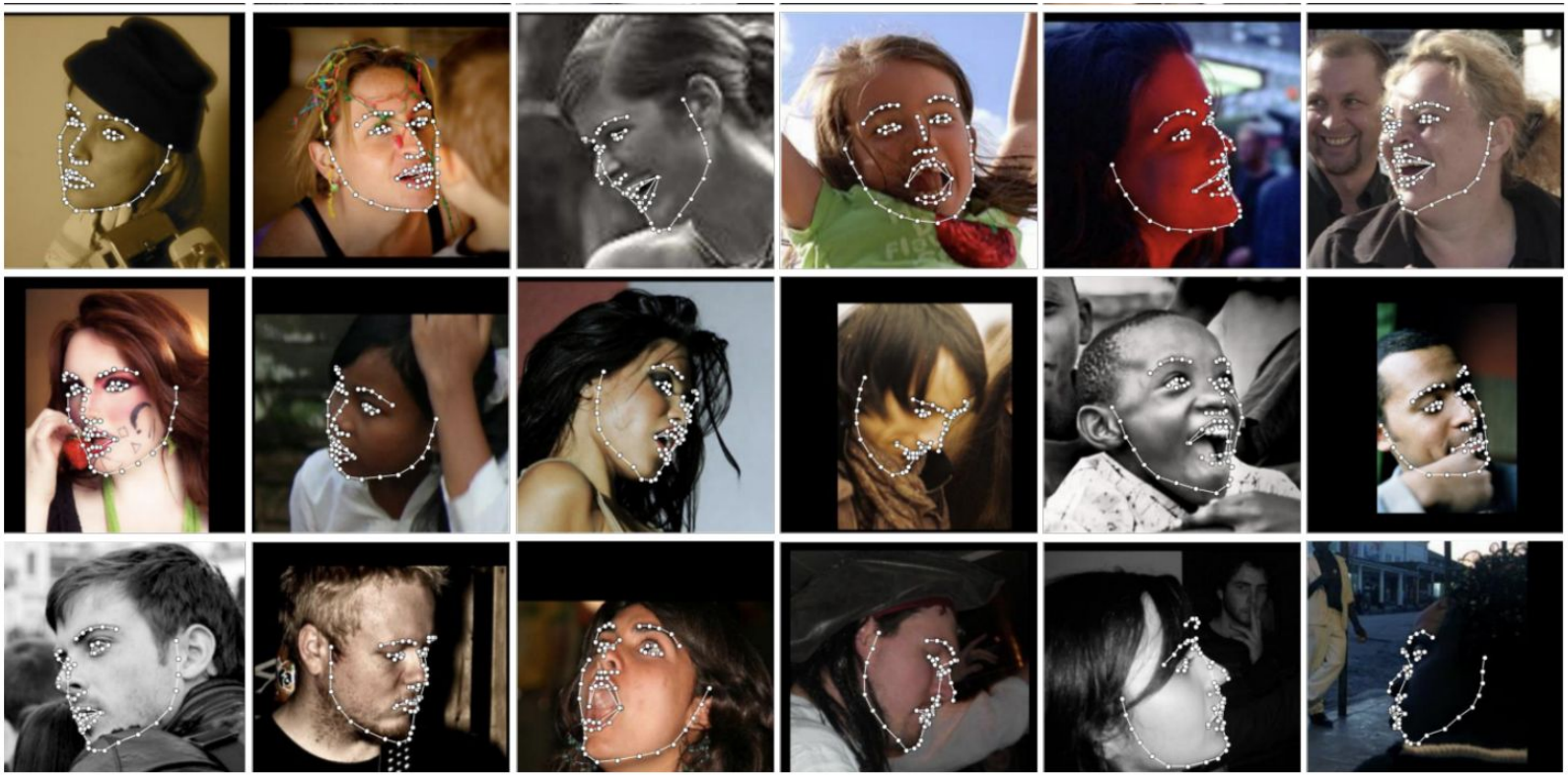


Figure 1: The Face Alignment Network (FAN) constructed by stacking four HGs in which all bottleneck blocks (depicted as rectangles) were replaced with the hierarchical, parallel and multi-scale block of [7].

FAN (Alineamiento de cara)



Bibliografía

- Deep Learning, Ian Goodfellow and Yoshua Bengio and Aaron Courville - Capítulo
- Neural Networks and Deep Learning. Charu C. Aggarwal

Papers:

U-net https://arxiv.org/pdf/1505.04597.pdf

Yolo https://arxiv.org/pdf/1506.02640.pdf

R-CNN https://arxiv.org/pdf/1311.2524.pdf

FAN https://arxiv.org/pdf/1703.07332.pdf

Hourglass https://arxiv.org/pdf/1603.06937.pdf